A Certificate-Based Branching Analysis for MaxSAT Parameterized by the Number of Clauses

Abstract

We present a pipeline for constructing and checking local branching analyses for exact MaxSAT. Starting from a partially specified clause neighborhood, the generator distinguishes empty and nonempty clause tails, exposes further variables, and searches for branching strategies. It combines assignment enumeration, removal of dominated alternatives, grouping by variable substitutions, reductions, and recursive analysis of the resulting formulas. The output is a certificate recording both the coverage of the case analysis and the transformations used in each case. A separate checker reconstructs the refinements, checks local optimum identities, and evaluates branching inequalities with exact rational arithmetic.

At a fixed target q, continuation choices are compared by the weight $W_q = \sum_i q^{-d_i}$ of their accumulated measure decreases. Its additivity permits independent continuation selection within each group. We apply the pipeline to the $(3, 2)$-literal bottleneck in the $O^*(1.2886^m)$-time algorithm of [1], where m is the number of clauses. Combining generated strategies with branching lemmas from that algorithm, the local analysis reaches the target $q = 1.2872$. We prove the soundness of the certificate rules and state the conditions under which this local result yields an $O^*(1.2872^m)$-time algorithm. The contribution is a compositional, machine-checkable analysis pipeline relative to a fixed library of imported branching lemmas, with the improved local target serving as its case study.

1 Introduction

Exact algorithms for unweighted MaxSAT seek an optimal assignment while bounding the size of the search needed to find it. The number of clauses m is a useful measure for this purpose: a branch can be charged for the clauses it removes, and several consecutive transformations can be analyzed through their accumulated decreases. The notation O^* suppresses factors polynomial in the input length.

Clause-measure analyses have improved through increasingly detailed treatment of low-degree variables. Bansal and Raman [7] obtained the bound $O^*(1.3413^m)$, Chen and Kanj [8] improved it to $O^*(1.3248^m)$, and Xu et al. [9] reached $O^*(1.2989^m)$ using resolution and domination. The algorithm of [1], which we call the *baseline algorithm*, further improved the bound to $O^*(1.2886^m)$ with polynomial space. Table 1 places our target in this line of results.

Year	Analysis	Running-time bound
1999	Bansal and Raman [7]	$O^*(1.3413^m)$
2004	Chen and Kanj [8]	$O^*(1.3248^m)$
2019	Xu et al. [9]	$O^*(1.2989^m)$
2022	Baseline algorithm	$O^*(1.2886^m)$
This work	Certificate-based replacement of Step 5.4	$O^*(1.2872^m)$

Table 1: Selected clause-measure bounds for unweighted MaxSAT. The last row is the global consequence under the certificate and imported-result premises of theorem 5.

1.1 The $(3,2)$-literal bottleneck

The baseline algorithm applies reductions and branching cases in a fixed order. Its Step 5 handles a literal x occurring in three clauses and whose complement occurs in two, with incident clauses

$$(x \vee C_1)(x \vee C_2)(x \vee C_3)(\bar{x} \vee D_1)(\bar{x} \vee D_2). \tag{1}$$

Step 5.4, where D_1 and D_2 contain only degree-five variables, gives the limiting branching vector $(6,6,5,5)$. Table 2 summarizes the baseline cases and identifies the one we replace. The other cases already meet the target $q = 1.2872$: in particular, $(11,3,3)$ has branching factor approximately 1.287190881. Improving Step 5.4 to this target is therefore sufficient to improve the global bound, provided that the replacement covers the case and preserves an optimal solution under the baseline algorithm's hypotheses.

Step	Literal or case type	Branching vector	Base
1	Degree at least six	$(6,1)$	1.2852
2	$(i,1)$ non-singleton, $i \geq 3$	$(3,3)$	1.2600
3	Degree three	$(8,1)$	1.2321
4	Unit-clause case	$(5,2)$	1.2366
5.1	$(3,2)$, unit-clause case	$(4,2)$	1.2721
5.2	$(3,2)$	$(11,3,3)$	1.2872
5.3	$(3,2)$	$(6,1)$	1.2852
5.4	$(3,2)$ **bottleneck**	$(\mathbf{6},\mathbf{6},\mathbf{5},\mathbf{5})$	**1.2886**
5.5	$(3,2)$	$(11,10,4,3)$	1.2801
6.1	$(2,2)$	$(4,2)$	1.2721
6.2	$(2,2)$	$(6,1)$	1.2852
6.3	$(2,2)$	$(11,3,3)$	1.2872
6.4	$(2,2)$	$(18,18,11,11,11,11,4,4)$	1.2870
7	$(3,1)$- or $(4,1)$-singletons	Set Cover Problem	1.2576

Table 2: Baseline cases, adapted from Table 3 of [1]. Bases reproduce the source's displayed values; for Step 7 the base is that of the terminal algorithm. We replace the highlighted Step 5.4 by a family of local strategies checked at $q = 1.2872$.

The generator starts from the five-clause neighborhood in equation (1), with partially specified clause tails. It allows each tail to be empty, so its root family includes cases covered by neighboring steps as well as the intended bottleneck. The structural restrictions used when further variables are exposed come from the ordered baseline algorithm. The exact root and allowed degrees are given in section 6.1; their coverage of Step 5.4 is established in section 6.

1.2 Automating a compositional analysis

A good branching rule for an exact algorithm must do two things: preserve an optimal solution among its children and reduce the instance enough to meet the desired running-time bound. Both tasks become harder when a useful rule requires several rounds of branching and reduction. The argument must also cover every possible local configuration, including configurations whose clauses contain variables outside the displayed neighborhood.

The bottleneck supplies a concrete setting for automating this analysis: the target is fixed, the initial neighborhood is small, and the surrounding algorithm supplies degree restrictions. A useful local strategy may group assignments by substitutions and then reduce or branch on the residual formulas. Its quality depends on the composition of all these steps, not just on the first branching vector. Structural case coverage and recursive strategy selection must therefore be tracked together.

Our pipeline separates two trees that play different roles. A *refinement tree* distinguishes possible clause neighborhoods. For each of its terminal cases, a *strategy tree* describes how

to reduce or branch on an instance in that case. The generator builds these trees from an analysis script. Within each selected case, it enumerates assignments, removes dominated choices, groups compatible assignments, and analyzes the residual formulas. A separate checker reads the resulting trees without repeating this search.

We select continuations using their weight at the fixed target, $W_q(v) = \sum_i q^{-d_i}$, where $v = (d_1, \ldots, d_r)$ is the vector of accumulated clause decreases. A preceding decrease scales this weight by q^{-d}, and recursive alternatives contribute additively. Consequently, each group can retain its best available continuation independently of the others; the search then chooses a minimum-weight cover of the remaining assignments. Ranking continuations by their standalone branching factors does not have this property, because their weight curves can cross. Lemma 3 and section 5.2 give the formal argument and its use in the generator.

The key verification step is simple. For each possible truth-value combination of the unknown clause tails, the checker computes the best local score before and after a proposed transformation. The scores must agree after accounting for the clauses already satisfied. This single test handles reductions and grouped branches through the same interface. Recurrence bounds are checked separately from these objective-value identities.

Contributions.

1. We give a pipeline that combines structural refinement with recursive search for local MaxSAT strategies, including assignment dominance, substitution-based grouping, and reductions of the resulting children. Fixed-target weights permit independent continuation selection and minimum-weight selection of possibly overlapping groups.

2. We define certificate rules for coverage, local transformations, and accumulated measure decreases, and prove their compositional soundness.

3. We apply the pipeline at the rational target $q = 1.2872$ and give an explicit successful strategy from the generated analysis. We separate the generated transformations from the imported results used to derive the running-time bound.

1.3 Related work and positioning

Automatic branching analysis has a substantial history. Fedin and Kulikov [2] generated running-time proofs from supplied simplification rules and reported experiments for SAT, MaxSAT, and bounded-occurrence MaxSAT. Kulikov [3] also automated the discovery of simplification rules and useful splittings for SAT and MaxSAT. Skjernaa's framework [4, Chapter 8] refines formula patterns, searches for reductions, and considers branching on variable values and on whether pairs of variables are equal or different. Thus automatic case analysis, reduction discovery, and branching beyond individual assignments are established ideas.

More recently, Červený and Suchý [5] generated parameterized branching algorithms for d-Path Vertex Cover. Gao et al. [6] formulate local branching-rule selection for Maximum Independent Set as a weighted set-cover problem. These works also motivate searching for a collection of alternatives rather than choosing a single branching variable.

Our focus is the integration of clause-tail refinements, recursive analysis of substitution-generated residual CNFs, and a separate checker for both coverage and local optimum identities. In particular, the certificate records actual child formulas and their composition, so a newly added search heuristic does not require a corresponding heuristic-specific verification rule. We present this combination for the baseline bottleneck; we do not claim that deeper search, dominance, or grouping alone is new.

Related resolution techniques appear in Chen, Xu, and Wang [10]. Other parameterizations, including those studied by Bliznets and Golovnev [11], lead to different bounds. General background on branching recurrences is provided by Fomin and Kratsch [12].

Finally, proof logging for MaxSAT can certify optimum-preserving preprocessing, including through the VeriPB and CakePB systems [13]. Our certificate has a different target: a local case analysis and its recurrence for a family of instances. Its semantic checks enumerate small local truth tables, while pseudo-Boolean proof logging supports reasoning about individual, potentially large solver instances.

Organization.

Section 2	Local notation and target weights.
Section 3	Pipeline and worked example.
Section 4	Certificate rules, composition, and soundness.
Section 5	Automated search and independent checker.
Section 6	Generated analysis, global bound, and artifact.
Section 7	Conclusion.
Appendices A and B	Normalization conventions and analysis script.

2 Preliminaries

We use standard unweighted MaxSAT terminology as in [1]. A formula F is a multiset of clauses: equal clauses are retained with multiplicity. We assume the normalization described in appendix A, and write $m = |F|$ and

$$\mathrm{OPT}(F) = \max_{\alpha} \mathrm{sat}(F, \alpha),$$

where α ranges over all assignments and sat counts satisfied clauses with multiplicity. We describe transformations of the optimum directly. Throughout, $F \uplus G$ denotes multiset union: clause multiplicities add.

A literal x has degree (i, j) if it occurs in i clauses and its complement $\bar{x}$ in j clauses. The complementary literal has degree (j, i). An $(i, 1)$-literal is a *singleton* when its unique negative occurrence is the unit clause $(\bar{x})$.

2.1 Templates, scores, and measure decreases

A *template* P is a multiset of displayed clauses. It names selected variables explicitly and uses a *tail* ? for the as-yet-undisplayed part of a clause. A $?^+$ tail contains at least one literal, whereas a ? tail may be empty. These marks describe clause syntax, not truth values; even a nonempty tail may evaluate to false. Subscripts distinguish tails, and repeated identifiers denote the same disjunction. All occurrences of a selected variable are displayed. Clauses written consecutively form a multiset, so repeated clauses count separately.

A complete formula *matches* P when it can be written as $F = P[T] \uplus E$. Here T replaces each tail by a disjunction of literals on unselected variables, and E contains the remaining clauses. Even when a clause occurs in both parts, its multiplicities add. No selected variable occurs in T or E. Matching also respects any degree and singleton restrictions stated for the template. For a Boolean vector τ indexed by the tails, define the *local score*

$$B_P(\tau) = \max_{\alpha} \mathrm{sat}(P, \alpha, \tau), \tag{2}$$

where α assigns the selected variables and each tail is treated as a Boolean input. A clause made true by a tail counts as one satisfied clause. Different tails can share hidden variables in a complete instance. Checking all Boolean vectors τ includes every realizable combination, and may therefore reject a valid transformation, but cannot accept an invalid one for this reason.

A correct branching step produces formulas F_i and nonnegative integer *offsets* s_i such that

$$\mathrm{OPT}(F) = \max_i \big(s_i + \mathrm{OPT}(F_i)\big).$$

Its *measure decreases* are $d_i = |F| - |F_i|$. These need not equal the offsets: an empty false clause contributes to d_i but not to s_i. For example, assigning $x = 1$ in $(x)(\bar{x})$ gives $s = 1$ and $d = 2$. The generator represents such steps locally by transforming templates; lemma 2 connects a checked local identity to the corresponding full formulas F_i.

2.2 Fixed-target weights and recurrences

For a target $q > 1$ and a nonempty integer vector $v = (d_1, \ldots, d_r)$, define its *fixed-target weight*

$$W_q(v) = \sum_{i=1}^{r} q^{-d_i}. \tag{3}$$

Repeated entries are counted separately. A vector of positive integer decreases is *q-admissible* when

$$W_q(v) \leq 1. \tag{4}$$

We use the target $q = 1.2872$ and check this inequality exactly. A local recursive call with no additional decrease has vector (0) and weight one. Such a call can end a continuation after a positive incoming decrease; by itself it does not establish a progressing recursive step.

Lemma 1 (Branching recurrence)**.** *Suppose each recursive step has positive decreases satisfying equation (4), and the work at each node, including any intervening reductions, is polynomial in the input length. Then the running time is $O^*(q^m)$.*

Proof. Induction on m bounds the number of leaves by q^m: an internal node has at most $\sum_i q^{m-d_i} \leq q^m$ descendant leaves. Each root-to-leaf path has at most m recursive edges, since every decrease is positive. The number of nodes is therefore at most $(m + 1)q^m$. Polynomial work per node gives the claimed bound. □

3 Pipeline overview and a worked example

The refinement tree first asks structural questions. Is a chosen tail empty? If not, which degree can a literal in it have, and where can its other occurrences lie? Each answer gives a child template. The strategy search then asks an optimization question: which assignments or groups of assignments cover an optimal solution with a sufficiently good branching vector? Figure 1 shows how the script interleaves them.

```
frontier <- {root template}
for each command in the analysis script:
    if the command selects a refinement:
        replace selected unresolved templates by their children
    if the command starts a strategy search:
        for each selected unresolved template:
            if a strategy is found, record it and remove the template
require frontier empty
assemble the certificate; check coverage, identities, and vectors
```

Figure 1: The generation and checking pipeline. Refinements cover structural cases; a strategy search resolves cases when it finds a suitable branch or reduction. The script chooses the refinements and search budgets.

Example 1 (A grouped branch from the certificate)**.** Consider the following template, obtained by exposing a $(3,1)$-singleton y in the two negative clauses of the root:

$$\begin{aligned} P = {} & (x \vee ?_1)(x \vee ?_2)(x \vee ?_3) \\ & (\bar{x} \vee y \vee ?_4)(\bar{x} \vee y \vee ?_5)(y \vee ?_6)(\bar{y}). \end{aligned} \tag{5}$$

The four assignments to (x, y) need only two alternatives. The assignments $(0,0)$ and $(1,0)$ are grouped by fixing $y = 0$. When $y = 1$, changing x from 0 to 1 cannot lose a satisfied clause: the two negative x-clauses are already satisfied by y. Thus $(0,1)$ is dominated by $(1,1)$.

For the first alternative, let

$$H = (x \vee ?_1)(x \vee ?_2)(x \vee ?_3)(\bar{x} \vee ?_4)(\bar{x} \vee ?_5)(?_6).$$

The unit clause $(\bar{y})$ contributes offset 1, and all six clauses of H remain in the recursive formula, including the tail-only clause $(?_6)$. The second alternative satisfies six clauses and falsifies $(\bar{y})$, leaving no local clause. It has offset 6 and decrease 7. Consequently, for every Boolean tail vector τ,

$$B_P(\tau) = \max\{1 + B_H(\tau),\, 6\}. \tag{6}$$

In a complete instance $F = P[T] \uplus E$, the same identity gives

$$\mathrm{OPT}(F) = \max\{1 + \mathrm{OPT}(H[T] \uplus E),\, 6 + \mathrm{OPT}(E)\}.$$

Thus the external clauses remain in both recursive alternatives, including the alternative with no remaining local clause. The vector is $(1,7)$, and $q^{-1} + q^{-7} < 0.948 < 1$ at $q = 1.2872$.

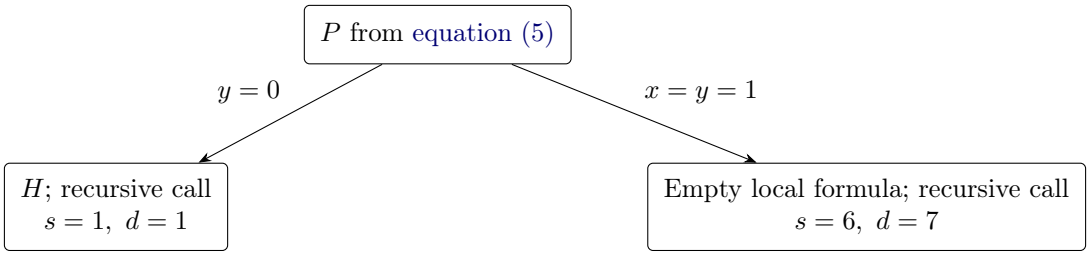


Figure 2: A strategy extracted from the existing certificate, with variable and tail names relabeled. The external part of the instance remains to be solved even when the local formula is empty.

This example also explains why the generator constructs residual formulas. A grouped alternative can retain a free variable, so further reductions and branching can improve its contribution to the final vector. The search can continue below H instead of treating the group as an indivisible leaf.

4 Templates and certificate rules

We now formalize the two checks used above: refinements cover the relevant instances, and strategies preserve their optimum. We use F for a complete recursive formula and P for its displayed template.

4.1 Unknown tails and the surrounding formula

For a strategy transformation, not every tail identifier of the parent need occur in a child; every identifier that does occur must come from the parent. The checked score identity justifies any disappearance of local clauses or tails. Structural refinement has a different role. When a variable

is exposed from a tail, the child's tail describes the remainder after that exposure, and the displayed fragment may acquire further clauses from E. Matching is always interpreted on the same complete instance, even though its displayed fragment changes.

The local score $B_P(\tau)$ from equation (2) lets the checker compare such transformations without constructing a complete formula for each choice of external clauses.

4.2 Structural refinements

Refinements distinguish formula structure without making a Boolean assignment. The two basic operations are shown in figure 3.

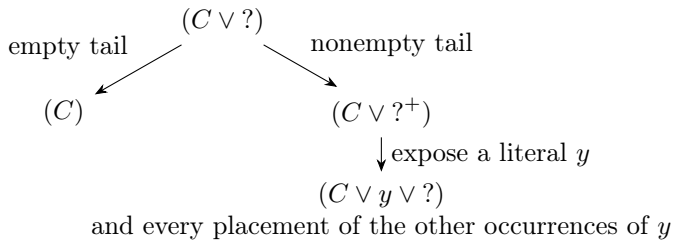


Figure 3: Topological refinement nodes. If C is nonempty, the left child is an ordinary clause, not an empty clause. After exposing one literal from $?^+$, the remaining tail is ? and may be empty.

Tail split. Replace a chosen ? by the two cases "empty" and $?^+$. These cover every concrete tail. This is recorded as `tail_empty_or_nonempty`.

Variable exposure. An analysis specifies a finite set of *allowed oriented degrees* (i, j) for the literal selected from a clause tail. The orientation counts occurrences of that literal first and its complement second. In our case this set consists of five classes, listed in section 6.1. Its completeness follows from the structural assumptions inherited from the baseline ordered algorithm, not from a bound on clause width. Choose a literal from a nonempty tail and name it y. For each degree in this set, enumerate all placements of y and $\bar{y}$ in existing clause tails, and add clauses for the remaining occurrences. The selected clause must contain y; singleton restrictions fix the unique negative occurrence to a unit clause. The remaining tail in each affected clause may be empty. Each concrete occurrence pattern is among these choices, provided the degree list covers the literals that can be selected.

The format combines a tail split and these exposures in `expose_at_clause`: an empty child is included for ? and omitted for $?^+$. Exposure is restricted to templates with distinct, clause-local single tails. An initial variable declaration constructs the root family; it is not a refinement of arbitrary formulas.

The coverage requirement can be stated without a separate semantics for fragments: *every complete instance matching the parent and its structural restrictions must match at least one child.* Exposing y merely moves all its occurrences into the displayed fragment of that same instance. The checker regenerates the child templates, compares them up to renaming, and rejects missing, extra, or duplicate cases. In format version 2 the allowed degree list is recorded in the exposure node. Its completeness under the intended structural restrictions is a mathematical premise, whereas faithful enumeration of its incidences is checked computationally.

4.3 Local strategies

The basic certificate operations are refinement, decomposition, recursive call, and imported lemma application. Refinements may occur within a strategy as well as above its root. Proof-backed

local lemmas and partially expanded imported lemmas extend these operations as described below. Decompositions have two forms.

Assignment decomposition. The alternatives assign a selected variable x to 0 and 1. The checker simplifies both children and recomputes their offsets and decreases. Suppose an assignment leaves a displayed fragment P_i and transfers a multiset R_i of tail-only clauses to the surrounding formula. Then

$$F_i = P_i[T] \uplus E \uplus R_i[T], \qquad d_i = |P| - |P_i| - |R_i|. \tag{7}$$

Satisfied clauses contribute to s_i; empty false clauses contribute only to d_i. The transferred clauses remain part of the recursive instance. For example, setting $x = 0$ in $(x \vee ?)(\bar{x})$ transfers $(?)$ and gives $s = d = 1$, even though no selected variable remains.

Semantic decomposition. The certificate supplies children $H_1, \ldots, H_r$ and offsets s_i. The checker accepts the transformation when

$$B_P(\tau) = \max_{1 \le i \le r} \big(s_i + B_{H_i}(\tau)\big) \quad \text{for every } \tau. \tag{8}$$

Every child uses only parent tails and selected or fresh local variables. Here the surrounding formula is unchanged, so tail-only clauses that remain must be included in H_i, as in example 1. The decrease is $d_i = |P| - |H_i|$. A one-child decomposition describes a reduction; its offset can also be inferred if $B_P - B_{H_1}$ is a constant nonnegative integer.

Lemma 2 (Replacement inside a formula)**.** *Suppose equation (8) holds and the local variables of the parent and children occur nowhere in the surrounding formula or the concrete tails. Then replacing $P[T]$ by the alternatives $(H_i[T], s_i)$ preserves the optimum of $P[T] \uplus E$.*

Proof. Fix an assignment β to all external variables. It gives tail values τ_β and satisfies some $e(\beta)$ clauses in E. The parent optimum is

$$\max_\beta \big(e(\beta) + B_P(\tau_\beta)\big) = \max_i \left(s_i + \max_\beta \big(e(\beta) + B_{H_i}(\tau_\beta)\big) \right),$$

using equation (8). The expression on the right is the best shifted optimum of a child in the same surrounding formula. □

Recursive call and imported lemma. A `call` ends the local analysis and invokes the global algorithm on the remaining complete formula. The decrease accumulated along the path is the decrease of this leaf. An imported lemma application, encoded as `assumption`, supplies a named branching vector instead. The checker records its name, formula, vector, and tree path; its mathematical justification comes from the cited branching result and its applicability to the current instance. The generated certificate uses the baseline Lemmas 2–4, together with the repeated Lemma 3 argument in Step 5.5 of that algorithm. In particular, we use Lemma 3 of [1], a refinement of Lemma 9 of [9]. These numbers refer to the source results, not to the numbered statements of this paper. Table 3 gives the correspondence to the certificate names; Step 5.1 is supplied by a checked local proof.

Composing two applications of Lemma 3 from [1]. Consider the branching case of that source lemma for a $(2, 1)$-singleton y_1. Its resolution child replaces

$$(y_1 \vee A)(y_1 \vee B)(\bar{y}_1) \quad \text{by} \quad (A)(B)$$

with offset and decrease one. In this case A and B are nonempty; short clauses that trigger the source reductions are handled before this branching case. Removing y_1 from these two clauses

Certificate name	Mathematical dependency
`lemma2`	Lemma 2 for an $(i, 1)$ non-singleton, imported there from [9].
`lemma3`	Lemma 3 for a degree-three variable, refining Lemma 9 of [9].
`lemma4`	Lemma 4 for specified unit occurrences.
`double_lemma3`	The composition of Lemma 3 used in its Step 5.5, with the second singleton preserved in the resolution child.
`step5.1`	Step 5.1, the unit-clause case for a $(3, 2)$-literal, supplied by a checked assignment proof.

Table 3: Branching dependencies from [1] and their certificate names. Lemma and step numbers in this table belong to that source. Imported uses require the source hypotheses, including any reduction-or-branch alternative and the claimed decreases in the complete instance. Opaque imported applications are logged; proof-backed applications are expanded and checked.

preserves every occurrence of any other variable, since clause multiplicities are retained. In particular, a distinct $(2, 1)$-singleton y_2 retains its two positive occurrences and its negative unit clause. When the branching case of the same source lemma applies again, the two nested vectors $(1, 8)$ give

$$(8,\ 1+8,\ 1+1) = (8, 9, 2).$$

This is the composition used in Step 5.5 of the baseline algorithm. The reduction alternatives of the imported lemmas must still be handled as reductions when they apply; the calculation above describes their nested branching case.

Proof-backed and partially expanded lemmas. The current implementation expands Step 5.1 into a checked assignment proof. A `local_context` node embeds its local template in the current formula: all incident clauses of the kept variables are retained, other literals are bound to tail expressions, and unrelated clauses stay in the surrounding formula. The checker validates this embedding, clause multiplicities, tail constraints, and freshness, then checks the attached proof. The embedding contributes zero offset and zero measure decrease. Its leaf vector is recomputed from the proof, and only the complete enclosing strategy must meet the target.

A `lemma3_split` node instead exposes the resolution child of the same source Lemma 3 and checks a continuation T on that child. For incident clauses $(x \vee C_1)(x \vee C_2)(\bar{x} \vee D)$, this child replaces them by $(C_1 \vee D)(C_2 \vee D)$, with offset and decrease one. Writing $a = \max\{8, 7 + 2|D|\}$ for the source's long-branch guarantee, the node is bounded at the fixed target by

$$\max\{q^{-1},\ q^{-a} + q^{-1} W_q(v(T))\}. \tag{9}$$

The maximum is necessary because the source lemma permits either a reduction of at least one clause or the displayed branching. That unspecified reduction need not produce the resolution child, so it cannot inherit T. The checker validates the displayed resolution and its continuation; the long-branch guarantee and reduction-or-branch alternative remain imported premises. This operation is unavailable when imported assumptions are disabled.

4.4 Composition and soundness

A decomposition edge with decrease d followed by a child vector $(e_1, \ldots, e_t)$ contributes $(d + e_1, \ldots, d + e_t)$ to the enclosing strategy. Concatenating the vectors of all alternatives gives its leaf vector. The complete accumulated vector in every structural case must have positive entries and satisfy equation (4); an intermediate reduction may have decrease zero. Refinement children are separate structural cases, so their vectors are not concatenated. At an outer refinement each case is checked separately. Within a strategy their contribution is bounded by the maximum of the case weights; an incoming decrease scales this maximum just as it scales a single continuation.

Let $d + v$ denote the vector obtained by adding d to every entry of v, and let $v\|u$ denote concatenation. Directly from equation (3),

$$W_q(d + v) = q^{-d} W_q(v), \qquad W_q(v\|u) = W_q(v) + W_q(u). \tag{10}$$

Thus recursive alternatives add their shifted weights, whereas structural cases contribute their worst weight. Both operations preserve an improvement of any child weight at the fixed target.

Lemma 3 (Independent continuation selection)**.** *Fix $q > 1$ and a decomposition with incoming decreases d_g for its alternatives $g = 1, \dots, k$. For each alternative let $\mathcal{C}_g$ be a finite nonempty set of available correct continuations, each with positive accumulated decreases after its incoming edge. Suppose these choices are independent. Then the minimum weight of the composed strategy is*

$$\min_{(S_g) \in \prod_g \mathcal{C}_g} \sum_g q^{-d_g} W_q(v(S_g)) = \sum_g q^{-d_g} \min_{S_g \in \mathcal{C}_g} W_q(v(S_g)). \tag{11}$$

Proof. By equation (10), the left-hand objective is the weight of the composed leaf vector. Each coefficient q^{-d_g} is positive, and each summand depends only on S_g. Choosing a minimizing continuation in every set $\mathcal{C}_g$ attains the right-hand side; every other choice has weight at least that value. □

The statement concerns the available continuations at a fixed target. It does not assert that a bounded search discovers all possible strategies.

Theorem 4 (Local certificate soundness)**.** *Let a finite certificate pass the refinement, decomposition, and vector checks above at target q, with positive accumulated call decreases in every structural case. Suppose its exposure degree lists are complete under the stated structural restrictions, and each recorded imported lemma admits a correct recursive strategy with at least the recorded decreases for every complete instance to which it is applied. Then every such instance matching the root has an optimum-preserving local strategy whose accumulated decreases are q-admissible.*

Proof. Induct over the certificate. A tail split covers its parent, and variable exposure enumerates every permitted incidence pattern. Thus at least one refinement child applies to each matching instance. Assignment decomposition is exhaustive, with the clause accounting in equation (7). Semantic decomposition preserves the optimum by lemma 2. Calls end a local path, and imported applications use the stated hypothesis. Checked local-context embeddings preserve the surrounding formula; partially expanded imports use the checked continuation and the source alternatives in equation (9). Edge decreases telescope along each path; larger actual decreases can only improve equation (4). Taking a maximum at an internal refinement bounds every structural case by equation (10). The check at each strategy root establishes the claimed recurrence. □

5 Automated search and implementation

The C++ generator maintains a frontier of unresolved templates. The analysis script chooses which tails to expose and when to run a search pass. Each pass first tries a cheap enumeration-based strategy, then applies the requested stronger analysis to the remaining templates. Successful cases are removed from the frontier and their proof trees are logged. This division lets a short script direct the structural analysis while the program searches the local strategies automatically.

5.1 Dominance, grouping, and recursive search

For a selected set of variables, the generator enumerates assignments and records which clauses are satisfied, which become empty, and which remain unresolved. It removes an assignment if

another choice is at least as good for every possible tail valuation. The implementation uses clause masks for these comparisons and records them as *subsumptions*. This is dominance among assignment alternatives, not unrestricted deletion of subsumed CNF clauses in MaxSAT.

The surviving assignments must be covered by groups. Each group is represented by substituting each selected variable with a constant or a signed fresh variable:

$$x_j \longmapsto 0,\ 1,\ u_1,\ \bar{u}_1, \ldots, u_t, \bar{u}_t. \tag{12}$$

For example, $\{(0,0),(1,0)\}$ is represented by $(x,y) = (u,0)$, and $\{(0,1),(1,0)\}$ by $(x,y) = (u,\bar{u})$. Substitution produces an ordinary residual CNF to which reductions and further branching can be applied. The group must be exactly the image of the substitution; arbitrary affine constraints such as $x_3 = u_1 \oplus u_2$ are outside this representation. The generator enumerates substitution images over all assignments, so a candidate may also contain dominated assignments. It retains candidates covering at least one surviving assignment and materializes their complete images. Selected groups may overlap; all their recursive alternatives and costs remain in the resulting strategy.

The recursive variable search tries each available pivot and analyzes both assignment children up to the requested depth. For each child it may either stop with a call or retain its deeper analysis. Thus a useful strategy can be asymmetric. Grouped residuals also receive reductions and child analysis; the group pass includes an ordinary one-level variable search as a baseline. The depth parameters bound the search for an assignment tree, not the depth of the final global algorithm.

Reductions that remove clauses can end in a recursive call. The implemented zero-clause-decrease reductions eliminate a variable before continuing locally. Assignment depth limits and these decreases make each such local search finite. The outer refinement schedule is supplied by the script; the pipeline makes no completeness claim about the search for a strategy at an arbitrary smaller target.

5.2 Strategy selection at a fixed target

The generator compares candidates by W_q at the selected target, including their incoming decreases. By lemma 3, each group can retain one minimum-weight available continuation without enumerating combinations of continuation choices across groups. A direct call has weight one, so a recursive child analysis is retained when it improves on that weight. This permits asymmetric strategies in which one assignment child continues locally and the other immediately invokes the global algorithm.

Why the branching factor is insufficient. For a positive leaf vector with at least two entries, its branching factor $\rho(v) > 1$ is the unique solution of $W_{\rho(v)}(v) = 1$. Comparing these intersection points does not order the weights at another fixed target: two decreasing weight curves can cross. For example,

$$\rho((1,8)) \approx 1.232055 < 1.236506 \approx \rho((2,5)),$$

whereas at $q = 1.2872$,

$$W_q((1,8)) \approx 0.909568 > 0.886532 \approx W_q((2,5)).$$

An enclosing strategy with incoming decrease one and other leaves $(6,10)$ therefore gives opposite outcomes for the target test:

$$W_q((2,9,6,10)) \approx 1.006557 > 1,$$
$$W_q((3,6,6,10)) \approx 0.988660 < 1.$$

This arithmetic example illustrates composition; it does not require a separate formula template. Ranking by the standalone branching factor would discard the continuation that succeeds in this context. Earlier search could avoid this loss by retaining alternative continuations and enumerating their combinations; the fixed-target comparison removes that need.

Selecting a cover. Let A be the set of undominated assignments and let G range over the available substitution images. Give G the weight

$$c_q(G) = q^{-b_G} W_q(v(S_G)),$$

where b_G is its initial clause decrease and S_G its best available residual continuation with positive accumulated decreases. The generator solves the finite weighted cover problem

$$\min_{\mathcal{G}} \sum_{G \in \mathcal{G}} c_q(G) \quad \text{subject to} \quad A \subseteq \bigcup_{G \in \mathcal{G}} G. \tag{13}$$

Overlaps are allowed, and each selected group contributes its full weight. The solver memoizes sets of uncovered assignments; its search is exhaustive over the supplied candidates, although its worst-case cost is exponential. The winning cover is also compared with an ordinary one-level variable search.

Residual searches share a bounded cache whose key includes the search policy, depth, target, and applicability settings. Caching avoids repeated work on the same residual without changing its certificate. Floating-point weights guide the C++ search; branching factors are retained for reporting. Certificate acceptance uses exact rational weights and independently requires positive accumulated decreases. It certifies the selected strategy's correctness and target bound, not the optimality of the search result.

5.3 Certificate construction and checking

The generator logs the chosen refinements and strategies in JSON Lines. An assembler connects these records into one tree and rejects unresolved cases. The resulting JSON certificate is read by a separate Python checker. Formula normalization, refinement enumeration, assignment simplification, optimum tables, and rational vector arithmetic are implemented independently of the C++ search. Proposed reductions, dominance eliminations, and grouped branches share equation (8); local-context and partial-lemma nodes additionally carry the embeddings and continuations described in section 4.3. Internal structural cases are combined by the maximum of their exact fixed-target weights. Only the complete enclosing strategy is required to pass the target test.

For a template with a local variables, t tails, and representation length $\ell(P)$, a straightforward optimum-table computation takes $O(2^{a+t}\ell(P))$ time. This exponential work is confined to the finite local patterns used during offline verification. Once a certificate is fixed, its tables and tree sizes are constants with respect to the size of a future input instance.

The checker uses NumPy integer arrays for the score tables. It enforces a limit of 32767 clauses per local formula, stores scores in signed 16-bit arrays, and performs validated offset additions and differences in signed 32-bit arrays. Branching inequalities use exact rational arithmetic. The trusted implementation comprises the checker and its Python, JSON, and NumPy runtime; it is not a formally verified implementation of the calculus.

6 The generated analysis and its global consequence

6.1 Root family and analysis script

The root declared by `solve.txt` is

$$P_0 = (x \vee ?_1)(x \vee ?_2)(x \vee ?_3)(\bar{x} \vee ?_4)(\bar{x} \vee ?_5). \tag{14}$$

All five tails are initially arbitrary. The default exposure classes in this session are

$$(3,1)\text{-singleton}, \quad (2,2), \quad (3,2), \quad (2,3), \quad (4,1)\text{-singleton}.$$

Degrees are oriented toward the literal selected in the clause. The session first exposes y in a clause containing $\bar{x}$. It then exposes z in a clause containing $\bar{x}$ but not y, interleaving these refinements with strategy searches. The complete script and its command semantics are in appendix B.

The generated local analysis reaches $q = 1.2872$ using the imported results in table 3. Example 1 gives one of its explicit strategies, with no imported application in that strategy. For a complete certificate, every refinement leaf must have a strategy passing the local vector check. This is the computational result to which theorem 4 applies.

Bounded-width mode. With `-max-clause-size` k, the generated certificate proves only the declared local template family. The degree list stored at every variable exposure is an explicit premise of that family; enumerating placements proves coverage relative to this list, not completeness of the list for arbitrary k-MaxSAT. Clause width does not bound variable degree, and the 30-bit mask limit is only an implementation limit of the local search. A global k-MaxSAT bound would additionally require an ordered treatment of every omitted degree and structural case, a width-preserving recursive algorithm, its terminal algorithm, and the corresponding recurrences. Imported-lemma leaves are disabled by the generator and rejected by the checker in bounded-width mode, so each accepted bounded certificate contains only explicit checked transitions.

6.2 From a local strategy to a running-time bound

We retain the baseline reductions and all ordered cases except Step 5.4. Their connection to the generated analysis can be made explicit.

Coverage. At the start of Step 5, the preceding reductions and Steps 1–4 leave only $(2,2)$-variables, $(3,2)$-variables, and $(3,1)$- or $(4,1)$-singletons [1, Section 4.6]. A singleton's negative literal occurs only in its own unit clause, so it cannot be a hidden literal in a clause with a selected variable. Orienting a newly exposed literal toward the selected clause gives exactly the five classes in section 6.1. The initial five occurrences of x are precisely equation (14). Thus every instance reaching Step 5.4 matches the root under these restrictions. Structural refinements only reveal the same instance, so the degree restrictions remain valid throughout that refinement tree.

The other recurrences. For every branching vector outside Step 5.4 in table 2, exact evaluation at $q = 1.2872$ gives $W_q < 1$. The largest of these weights belongs to $(11,3,3)$: $W_q((11,3,3)) = q^{-11} + 2q^{-3} \approx 0.99997522 < 1$. Thus Steps 5.2 and 6.3 both satisfy the target despite the rounded base 1.2872 in the source table. In contrast, $W_q((6,6,5,5)) \approx 1.00567569 > 1$, isolating Step 5.4 as the case to replace. The terminal Step 7 takes $O^*(1.2576^m)$ time by Lemma 5 of [1], which is within the same bound.

Using a fixed certificate. The completed certificate is fixed independently of the input size. A refinement inspects whether a concrete tail is empty or selects a literal from it, scans its occurrences, and follows a matching child. Even enumerating all mappings of a fixed number of displayed variables takes polynomial time in the input length. The recorded assignments and substitutions are performed on the concrete clauses, with fresh local variables chosen outside the instance. Tail-only clauses remain in the recursive formula as in equation (7). A strategy has fixed finite depth and ends in calls whose accumulated decreases are positive. Imported routines use the polynomial operations of the source algorithm. These facts give polynomial time and

space for local matching, transformations, and dispatch, while all search for the certificate is performed offline.

Theorem 5 (Global consequence)**.** *Suppose the completed certificate at $q = 1.2872$ satisfies the premises of theorem 4, with the imported results applied under their source hypotheses. Replacing Step 5.4 of the baseline algorithm by its recorded strategies solves* MaxSAT *in $O^*(1.2872^m)$ time and polynomial space.*

Proof. Follow the ordered baseline algorithm, using the recorded local strategy when the replaced case is reached. Coverage permits this choice, and theorem 4 gives its correctness and branching inequality. The remaining cases satisfy the same target by table 2 and the bound for Step 7. The fixed-certificate dispatch described above and lemma 1 bound the branching work. Polynomial work at a leaf may be replaced by the Step 7 subroutine: a leaf of remaining measure m' has cost $O^*(1.2576^{m'}) \leq O^*(q^{m'})$, and the branching inequalities bound the sum of these leaf weights by q^m. A depth-first traversal has polynomial depth and stores polynomial information at each level. Together with the polynomial-space terminal subroutine, this proves the space bound. □

The imported lemmas are mathematical results used by this algorithm. Their certificate encoding determines the extent of machine checking: explicit transitions and recurrence arithmetic are checked, whereas imported applications are logged for inspection under their source hypotheses.

6.3 Reproducing the analysis

The generation session is run from the `maxsat_prover` source directory after building the C++ executable. The following commands generate, assemble, and check the certificate; the final command can also be run on an existing certificate without repeating generation.

```
make
./maxsat_prover test < solve.txt
python3 stitch_certificate.py pre_certificate.jsonl proof_tree.json
python3 verify_certificate.py proof_tree.json imported_lemmas.json
```

Generation recreates `pre_certificate.jsonl`. Verification writes the imported applications to `imported_lemmas.json`. Its verdict is `ACCEPT` when this log is empty; otherwise it reports

`ACCEPT RELATIVE TO IMPORTED LEMMAS.`

Generated artifact and measured costs. We reran the complete `solve.txt` session on October 14, 2026, using the fixed-target search described in section 5. All cases were resolved. The resulting version-2 certificate was assembled and independently checked, with verdict `ACCEPT RELATIVE TO IMPORTED LEMMAS`. Table 4 reports its structure and the measured costs of generation, assembly, and verification.

Measurements use `/usr/bin/time` and describe one run on an AMD Ryzen 7 5800U machine with 16 hardware threads and 15 GiB of memory, using Python 3.10.6, NumPy 2.2.6, and GCC 15.2.1. The executable was built with `-O3`; compilation is excluded from the generation time. Compared with the earlier recorded run, generation decreased from 38.011 to 1.72 seconds, assembly from 8.75 to 3.99 seconds, and verification from 30.72 to 13.46 seconds. These are single-run measurements of the full pipeline, not an isolated benchmark of one search optimization. Timing records and the implementation identifiers are retained with the artifact manifest.

The 1,723 imported applications consist of 1,502 uses of `lemma3`, 182 of `lemma2`, 24 of `lemma4`, nine of `double_lemma3`, and six of `lemma3_split`. The Step 5.1 application is checked explicitly and contributes no imported assumption. For exact artifact identification, the SHA-256 digests are

Artifact property	Value
Generation log	5,438 records; 9,377,807 bytes
Certificate	`maxsat-local-proof-v2`; 12,588,025 bytes
Tree	37,042 nodes; maximum depth 9
Refinement	52 nodes: 32 tail splits and 20 variable exposures
Strategies	7,120 roots
Decompositions	12,741 nodes: 8,396 semantic and 4,345 assignment
Calls	15,405 nodes
Imported applications	1,717 opaque nodes and six partially expanded nodes
Proof-backed embeddings	One `local_context` node for Step 5.1
Largest local formula	13 clauses
Generation	1.72 seconds wall-clock; 1.67 user, 0.03 system; 24.8 MiB peak RSS
Assembly	3.99 seconds wall-clock; 164.1 MiB peak RSS
Verification	13.46 seconds wall-clock; 186.3 MiB peak RSS

Table 4: Statistics and timings for the complete October 14, 2026 run. The independent checker reports `ACCEPT RELATIVE TO IMPORTED LEMMAS`.

```
pre_certificate.jsonl  ecd7029fc427f0fa2e858600b5d76fcf...
proof_tree.json         c5930bcd4bae29f6eda91aecbf648d72d...
```

The full digests are recorded in `artifact_manifest.txt`; the imported-lemma applications are listed in `imported_lemmas.json`.

7 Conclusion

We developed a pipeline for turning a partially specified MaxSAT neighborhood into a checked local branching analysis. Structural refinements cover the possible neighborhoods, and strategy search combines dominance, grouping, reductions, and recursive analysis of residual formulas. The certificate makes these stages explicit while reducing their verification to coverage, local optimum identities, and exact recurrence arithmetic. Fixed-target weights make continuation selection compositional: each group retains its best available continuation before the generator selects a minimum-weight cover. Structural cases use the worst weight, while recursive alternatives add their shifted contributions.

The $(3, 2)$-literal case demonstrates how this pipeline can meet the target 1.2872 and support a refinement of the baseline analysis. The local soundness theorem and the global consequence identify the mathematical dependencies needed to use the generated tree. The same separation allows the search to be extended while keeping a common, inspectable verification interface.

A Normalization conventions

Before applying the recursive algorithm, remove duplicate literals within each clause. Remove tautological clauses with an objective offset of one each, and empty clauses with offset zero. The initial measure m counts clauses after this normalization. Equal nonempty clauses are retained as distinct multiset occurrences. During a transformation, satisfied clauses contribute to both the objective offset and the measure decrease, whereas newly empty false clauses contribute only to the measure decrease. A tail-only clause remains in the recursive instance unless its emptiness or satisfaction is established.

B The analysis script

The complete generation session in `solve.txt` is:

```
set 1.2872

add x 3 2 a
addpos y
-x

branch
addpos z
-x
y
xiao_branch 1
xiao_branch 2
branch
empty_divide -1
branch

print 0
exit
```

The `add` command declares the root, with `a` allowing arbitrary tails. Each `addpos` reads a name, required clause literals, and excluded clause literals; blank lines delimit these inputs. It distinguishes empty tails and enumerates variable exposures. The `branch` command runs assignment enumeration and grouping. The two `xiao_branch` commands request recursive variable searches of depths one and two. `empty_divide -1` refines eligible tails in the unresolved frontier, and the last `branch` searches the resulting cases. The generator selects local strategies; the script controls the refinement schedule and search budgets.

References

[1] Mingyu Xiao. An exact MaxSAT algorithm: Further observations and further improvements. In *Proceedings of the Thirty-First International Joint Conference on Artificial Intelligence*, pages 1887–1893, 2022.

[2] Sergey S. Fedin and Alexander S. Kulikov. Automated proofs of upper bounds on the running time of splitting algorithms. In *IWPEC*, volume 3162 of LNCS, pages 248–259, 2004. doi:10.1007/978-3-540-28639-4_22.

[3] Alexander S. Kulikov. Automated generation of simplification rules for SAT and MAXSAT. In *SAT*, volume 3569 of LNCS, pages 430–436, 2005. doi:10.1007/11499107_35.

[4] Bjarke Skjernaa. *Exact Algorithms for Variants of Satisfiability and Colouring Problems*. PhD thesis, University of Aarhus, BRICS DS-04-5, 2004. Chapter 8. https://www.brics.dk/DS/04/5/BRICS-DS-04-5.pdf.

[5] Radovan Červený and Ondřej Suchý. Generating faster algorithms for d-Path Vertex Cover. In *WG*, pages 157–171, 2023. doi:10.1007/978-3-031-43380-1_12.

[6] Xuan-Zhao Gao, Yi-Jia Wang, Pan Zhang, and Jin-Guo Liu. Automated discovery of branching rules with optimal complexity for the Maximum Independent Set problem. arXiv:2412.07685, 2024. https://arxiv.org/abs/2412.07685.

[7] Nikhil Bansal and Venkatesh Raman. Upper bounds for MaxSat: Further improved. In *ISAAC*, pages 247–258, 1999.

[8] Jianer Chen and Iyad A. Kanj. Improved exact algorithms for MAX-SAT. *Discrete Applied Mathematics*, 142(1–3):17–27, 2004.

[9] Chao Xu, Wenjun Li, Yongjie Yang, Jianer Chen, and Jianxin Wang. Resolution and domination: An improved exact MaxSAT algorithm. In *Proceedings of the Twenty-Eighth International Joint Conference on Artificial Intelligence*, pages 1191–1197, 2019.

[10] Jianer Chen, Chao Xu, and Jianxin Wang. Dealing with 4-variables by resolution: An improved MaxSAT algorithm. *Theoretical Computer Science*, 670:33–44, 2017.

[11] Ivan Bliznets and Alexander Golovnev. A new algorithm for parameterized MAX-SAT. In *IPEC*, pages 37–48, 2012.

[12] Fedor V. Fomin and Dieter Kratsch. *Exact Exponential Algorithms*. Springer, 2010.

[13] Hannes Ihalainen, Andy Oertel, Yong Kiam Tan, Jeremias Berg, Matti Järvisalo, Magnus O. Myreen, and Jakob Nordström. Certified MaxSAT preprocessing. In *IJCAR*, volume 14739 of LNCS, pages 396–418, 2024. doi:10.1007/978-3-031-63498-7_24.